STARVNT DAY 6-7

UNIVERSAL EVENT EXECUTION

FIRST COMMERCIAL TRUST PROOF

2-DAY ENGINEERING EXECUTION MISSION

Working proof > feature count > documentation

Mission Owners: Yogesh - Vendor-Side Commercial Execution | Nesar - Central Automation + Execution Orchestration

"Event type changes configuration and policy; it does not create a new product architecture."

MISSION MAP

01	**Primary Objective**	What Day 6-7 must prove end-to-end.
02	**Architecture Principle**	Universal event core and shared commercial/execution spine.
03	**Role Separation**	Exact responsibility boundaries for Yogesh, Nesar and Core.
04	**Day 6 - Yogesh**	Vendor discovery, eligibility, cost, selection, enquiry and quote.
05	**Day 6 - Nesar**	Event orchestration, approval validation and idempotent booking request.
06	**Day 6 Joint Proof**	Golden test, attack suite and definition of done.
07	**Day 7 - Yogesh**	Vendor execution lifecycle and evidence-based completion submission.
08	**Day 7 - Nesar**	Payment verification, completion orchestration and settlement lifecycle.
09	**Day 7 Failure Tests**	Duplicate, cancellation, incomplete-service and retry recovery.
10	**Universal Proof**	Wedding, Puja, Corporate and Birthday on the same spine.
11	**Ownership + Stop Conditions**	Decision authority and hard stop rules.
12	**Final Acceptance**	The one demo and the engineering standard for success.

01 | Primary Objective

Day 6-7 is not a collection of independent Vendor, Automation, Booking, or Payment features. It is one integrated engineering mission.

MISSION

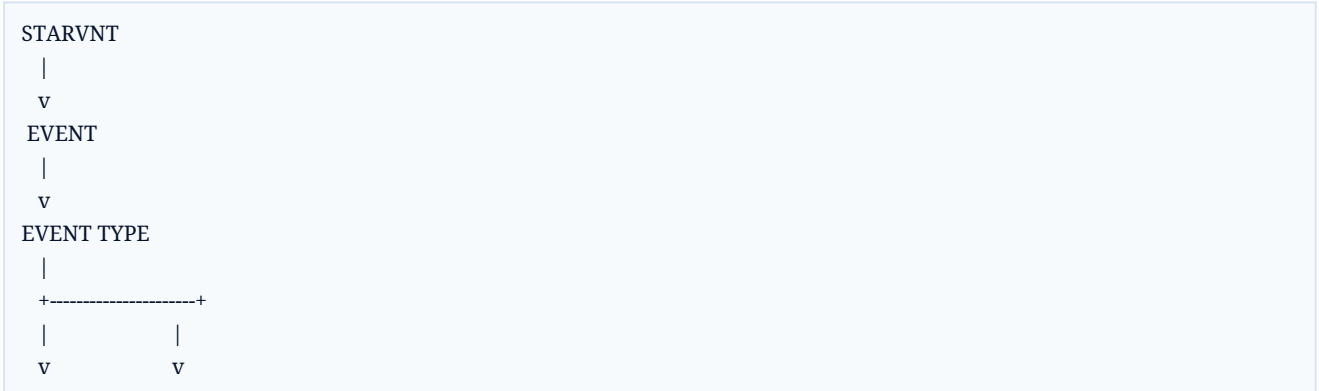
Prove that StarVnt can take a real customer requirement, identify a genuinely eligible vendor, validate the commercial cost, obtain approval, create a booking through Core, verify payment, manage vendor execution, verify completion, and safely reach settlement - using one universal architecture across different event types.

The target execution spine

```
CUSTOMER
 -> EVENT
 -> EVENT TYPE
 -> REQUIREMENT
 -> SERVICE LOCATION
 -> VENDOR DISCOVERY
 -> ELIGIBILITY
 -> VALIDATED TOTAL COST
 -> VENDOR SELECTION
 -> ENQUIRY
 -> QUOTE
 -> CUSTOMER APPROVAL
 -> BOOKING REQUEST
 -> BOOKING CORE
 -> PAYMENT CORE
 -> PAYMENT VERIFIED
 -> VENDOR EXECUTION
 -> COMPLETION SUBMITTED
 -> CORE VALIDATION
 -> COMPLETION VERIFIED
 -> SETTLEMENT ELIGIBLE
 -> VENDOR SETTLED
 -> EVENT READINESS
```

02 | Architectural Principle

The team must understand this before writing code: Event Type changes the requirement configuration and policies. It does not create a new commercial or execution system.



```
REQUIREMENT TEMPLATE    APPLICABILITY RULES
  |
  v
COMMON REQUIREMENT
  |
  v
COMMON COMMERCIAL SPINE
  |
  v
COMMON EXECUTION SPINE
```

Examples must enter the same architecture: Wedding -> Photography Requirement; Puja -> Decoration Requirement; Corporate -> Catering Requirement.

ARCHITECTURE RED LINE

A Wedding backend, Puja backend, Corporate backend, or Birthday backend is an architectural failure. Event type belongs in configuration, requirement templates, applicability rules, category capabilities, and completion policies - not in duplicate product systems.

03 | Role Separation

YOGESH - Vendor-Side Commercial Execution

- Vendor, service, capability, coverage, availability, capacity, eligibility and commercial data.
- Candidate discovery, cost calculation, vendor selection, enquiry and quote.
- Vendor execution lifecycle and completion submission/evidence.

Yogesh does not own Event authority, Booking authority, Payment authority, Completion verification authority, or Settlement authority.

NESAR - Central Automation + Execution Orchestration

```
Domain Events
  -> Trigger
  -> Condition
  -> Authorized Action
  -> Worker
  -> Retry
  -> Idempotency
  -> Audit
```

Nesar connects domain authorities. He does not become the authority for Event, Vendor, Requirement, Booking, Payment, or Settlement.

ORCHESTRATION RULE

Nesar moves an authoritative business event to the next authorized action safely. He must not invent business state or bypass domain authority.

04 | DAY 6 - YOGESH

Vendor Commercial Engine - Build + Prove

DAY 6 MISSION

Given a real StarVnt requirement, determine which vendors can actually fulfil it and which eligible option provides the lowest validated total cost under the applicable business rules.

Step 1 - Audit Existing Vendor Architecture

Do not start with code. Inspect the existing vendor system and identify the current source of truth.

```
Vendor
|-- vendorId
|-- organizationId
|-- category
|-- status
|-- baseLocation
|-- serviceAreas
|-- services
|-- capabilities
|-- availability
|-- capacity
|-- pricing
|-- quality
`-- eligibility
```

REUSE RULE

If the authoritative structure already exists: REUSE -> EXTEND -> INTEGRATE. Do not create NewVendorModel, VendorCommercialV2, VendorMatchingDB, VendorLocationDB, or VendorPricingV2 unless a real architectural gap is proven and approved.

Step 2 - Establish Vendor Coverage

```
VENDOR BASE LOCATION
      !=
SERVICE LOCATION

Requirement Service Location
      -> Vendor Service Coverage
      -> Can this vendor serve here?
```

A vendor based in Howrah may still be an eligible vendor for a New Town event if the vendor service coverage supports the service location. Base location is an input to logistics and cost, not an automatic rejection rule.

Step 3 - Consume the Core Requirement Contract

```
{
  "eventId": "EVT-001",
  "requirementId": "REQ-001",
  "organizationId": "ORG-001",
```

```
  "serviceCategory": "CATERING",
  "quantity": 300,
  "quantityUnit": "GUEST",
  "serviceLocation": {"city":"Kolkata","locality":"New Town","venue":"Convention Hall"},
  "eventDate": "2026-10-20",
  "duration": 8,
  "constraints": []
}
```

Yogesh must consume the Core requirement and must not fork it into another Requirement authority.

Step 4 - Candidate Discovery

```
REQUIREMENT
  -> SERVICE CATEGORY
  -> SERVICE LOCATION
  -> COVERAGE
  -> CANDIDATE PROVIDER
  -> VENDOR CANDIDATES
```

Candidate discovery is about not missing potentially serviceable vendors. Distance alone is not the winner rule.

Step 5 - Hard Eligibility

```
Category Match?
  -> Capability Match?
  -> Service Coverage?
  -> Availability?
  -> Capacity?
  -> Active?
  -> Minimum Quality?
  -> Requirement Constraints?
  -> ELIGIBLE / INELIGIBLE
```

Step 6 - Validated Total Cost

```
BASE SERVICE PRICE
+ TRAVEL COST
+ TRANSPORT
+ SETUP / DELIVERY
+ APPLICABLE ADDITIONAL CHARGES
+ OTHER VERIFIED COST
= VALIDATED TOTAL COST
```

The commercial comparison must use validated total cost, not merely advertised base price.

Step 7 - Quantity Must Affect Cost

```
CATERING
  -> 300 guests

PHOTOGRAPHY
  -> 8 hours + 2 photographers + 2 locations

TRANSPORT
```

```
  -> 50 passengers + 3 vehicles + 4 trips

DECORATION
  -> venue area + setup scope + theme/material requirements
```

Requirement quantity and dependencies must feed the calculation. Category complexity is expressed through requirement data and vendor capability, not duplicate engines.

Step 8 - Vendor Selection

```
ELIGIBLE VENDORS
  -> VALIDATED TOTAL COST
  -> LOWEST VALIDATED TOTAL COST
  -> TIE-BREAK POLICY IF REQUIRED
  -> SELECTED VENDOR
```

```
{
  "requirementId": "REQ-001",
  "selectedVendorId": "VEN-A",
  "selectionReason": "LOWEST_VALIDATED_TOTAL_COST",
  "baseCost": 45000,
  "travelCost": 2000,
  "additionalCost": 3000,
  "totalCost": 50000,
  "currency": "INR",
  "eligibility": "ELIGIBLE",
  "requiresCustomerApproval": true
}
```

NO OPAQUE AI WINNER

Do not replace commercial decision logic with an unexplained AI score such as "87.4". The selection must be machine-readable and explainable through eligibility, validated costs and explicit policy.

DAY 6 | YOGESH - Commercial Handoff

Step 9 - Enquiry

The selected vendor receives an enquiry containing enough context to understand what is required, where, when, for which event, and at what scope.

```
eventId
requirementId
organizationId
vendorId
serviceCategory
quantity
serviceLocation
eventDate
duration
constraints
```

Step 10 - Quote Lifecycle

```
DRAFT
  -> SUBMITTED
    -> APPROVED
    -> REJECTED
    -> EXPIRED
```

State transitions are enforced server-side. Example: APPROVED -> DRAFT must return HTTP 409. The frontend is never the authority for quote state.

Step 11 - Requirement Change

```
300 guests
  -> vendor selected
  -> requirement changes to 350 guests

REQUIREMENT_CHANGED
  -> OLD SELECTION = STALE
  -> RECHECK CAPACITY
  -> RECALCULATE COST
  -> RERANK
  -> NEW SELECTION / REVALIDATION
```

COMMERCIAL CORRECTNESS

A selection is valid only for the requirement that produced it. A changed requirement must trigger revalidation rather than silently reusing the old commercial result.

Yogesh Day 6 Live Proof

```
REAL EVENT
  -> REAL REQUIREMENT
  -> REAL SERVICE LOCATION
  -> REAL VENDOR CANDIDATES
  -> COVERAGE CHECK
  -> CAPACITY CHECK
```

-> AVAILABILITY CHECK
-> ELIGIBILITY
-> VALIDATED TOTAL COST
-> LOWEST SUITABLE VENDOR
-> ENQUIRY
-> QUOTE
-> CUSTOMER APPROVAL

05 | DAY 6 - NESAR

Central Automation + Booking Request Orchestration

DAY 6 MISSION

Turn authoritative business events into safe, validated, idempotent business actions through the existing Automation Engine.

Step 1 - Audit Existing Automation

```
Existing Events
Existing Triggers
Existing Conditions
Existing Actions
Existing Workers
Existing Outbox
Existing Retry
Existing Idempotency
Existing Audit
```

Reuse what already exists. Day 6 is not permission to create AutomationEngineV2, QuoteAutomationEngine, BookingAutomationEngine, or VendorAutomationEngine.

Step 2 - Domain Events

EVENT_CREATED
EVENT_TYPE_RESOLVED
REQUIREMENT_CREATED
VENDOR_MATCHED
ENQUIRY_CREATED
QUOTE_CREATED
QUOTE_SUBMITTED
QUOTE_APPROVED

Step 3 - Quote Automation

QUOTE_SUBMITTED
-> TRIGGER
-> CONDITION
-> AUTHORIZED ACTION
-> CUSTOMER NOTIFICATION

Automation consumes authoritative state; it does not invent business state.

Step 4 - QUOTE_APPROVED Validation

Correct Organization?
-> Authorized Actor?
-> Correct Requirement?
-> Quote belongs to Requirement?
-> Quote still valid?
-> Vendor still active?
-> Requirement unchanged?
-> Approval valid?
-> Booking already exists?
-> Idempotency check?
-> BOOKING_REQUEST

Step 5 - Duplicate Approval

```
QUOTE_APPROVED x 5
    -> IDEMPOTENCY
    -> ONE EFFECTIVE BOOKING_REQUEST
```

The system must never create five bookings because the same business outcome was delivered more than once.

06 | DAY 6 - JOINT GOLDEN PROOF

Primary Scenario

Event Type: Corporate Conference
Guests: 300
Service Location: New Town, Kolkata
Date: 20 October
Requirement: Catering
Quantity: 300 guests

EVENT
-> REQUIREMENT
-> SERVICE LOCATION
-> CANDIDATES
-> ELIGIBILITY
-> COST
-> SELECTION
-> ENQUIRY
-> QUOTE
-> APPROVAL
-> AUTOMATION
-> BOOKING_REQUEST

Observability Requirement

Everything must be observable in the appropriate system of record and operational logs: API response, persisted database state, domain event log, automation execution log, and audit trail.

Mandatory Attack Suite

TEST	ATTACK / CONDITION	EXPECTED RESULT
01	Fake vendor ID	BLOCK
02	Cross-tenant event / requirement access	BLOCK - no data leak
03	Required 300, vendor capacity 200	INELIGIBLE
04	Vendor cannot serve service location	INELIGIBLE
05	Expired quote approved	BOOKING BLOCKED
06	QUOTE_APPROVED repeated five times	ONE EFFECTIVE BOOKING_REQUEST
07	300 -> 350 guests after selection	OLD SELECTION STALE -> RE-EVALUATION

Day 6 Definition of Done

PASS CONDITION

Event Type -> Requirement -> Service Location -> Vendor Discovery -> Eligibility -> Validated Total Cost -> Best Suitable Vendor -> Enquiry -> Quote -> Approval -> Booking Request.

API = WORKING
DB = PERSISTED
EVENTS = WORKING
AUTOMATION = WORKING
IDEMPOTENCY = PROVEN
TENANT ISOLATION = PROVEN
AUDIT = PROVEN

Documentation alone = FAIL.

07 | DAY 7 - YOGESH

Vendor Execution + Completion

DAY 7 MISSION

Take the confirmed booking into the vendor execution lifecycle and submit verifiable completion evidence.

Step 1 - Booking Receipt

```
BOOKING_REQUEST
  -> BOOKING CORE
  -> BOOKING_CREATED
  -> VENDOR DASHBOARD CONSUMES BOOKING
```

Yogesh consumes the authoritative booking. He does not create the authoritative booking.

Step 2 - Vendor Confirmation

```
BOOKING_RECEIVED
  -> VENDOR_CONFIRMED
```

Vendor confirmation is a vendor-side action. It does not make the vendor a payment or settlement authority.

Step 3 - Execution Lifecycle

```
BOOKING_CONFIRMED
  -> PAYMENT_VERIFIED
  -> SERVICE_SCHEDULED
  -> SERVICE_STARTED
  -> MILESTONE
  -> COMPLETION_SUBMITTED
```

Step 4 - Completion Is Not a Boolean

DO NOT MODEL UNIVERSAL COMPLETION AS

serviceCompleted = true

Instead, the booking references a requirement and a completion policy that defines required milestones and evidence.

```
BOOKING
  -> REQUIREMENT
  -> COMPLETION POLICY
  -> REQUIRED MILESTONES
  -> EVIDENCE
```

Category-Specific Completion Policies

SERVICE CATEGORY	COMPLETION POLICY EXAMPLE
Photography	Coverage completed + required deliverables
Catering	Agreed meal/service completion
Decoration	Setup + event execution + dismantling
Transport	Required trip(s) + trip completion
Purohit	Agreed ceremony + service completion

Step 5 - Completion Submission

```
COMPLETION
  + REQUIRED MILESTONES
  + EVIDENCE
  + EXECUTION METADATA
  -> COMPLETION_SUBMITTED
```

Completion submission is not completion verification. The vendor can submit evidence; Core validates the outcome.

08 | DAY 7 - NESAR

Payment -> Execution -> Completion -> Settlement Orchestration

DAY 7 MISSION

Orchestrate the journey from verified payment to controlled execution, completion validation and settlement without becoming the financial authority.

Step 1 - Payment Core

```
BOOKING_CREATED
  -> PAYMENT_REQUIRED
  -> PAYMENT_INTENT
```

The payment amount comes from authoritative commercial/booking data. Frontend input cannot redefine payment authority.

Step 2 - Test Payment

```
PAYMENT_INTENT
  -> RAZORPAY TEST PAYMENT
  -> SERVER VERIFICATION
  -> WEBHOOK
  -> PAYMENT CORE
  -> PAYMENT_CONFIRMED
```

The browser/frontend is not payment authority.

Step 3 - Critical State Separation

```
PAYMENT_CONFIRMED      = TRUE
COMPLETION_VERIFIED    = FALSE
SETTLEMENT_ELIGIBLE    = FALSE
VENDOR_SETTLED         = FALSE
```

CORE TRUST RULE

PAYMENT_CONFIRMED != COMPLETION_VERIFIED != SETTLEMENT_ELIGIBLE != VENDOR_SETTLED. These are separate business states and must remain separate in persistence and APIs.

Step 4 - PAYMENT_CONFIRMED Orchestration

```
PAYMENT_CONFIRMED
  -> Validate Booking
  -> Correct Tenant?
  -> Payment belongs to Booking?
  -> Payment state valid?
  -> Idempotency
  -> Vendor Notification
  -> Audit
```

Step 5 - Vendor Execution

```
VENDOR_CONFIRMED
  -> SERVICE_SCHEDULED
```

-> SERVICE_STARTED
-> MILESTONE
-> COMPLETION_SUBMITTED

Nesar orchestrates domain transitions and actions; the vendor-side execution remains Yogesh territory.

Step 6 - Core Completion Validation

COMPLETION_SUBMITTED
-> Correct Booking?
-> Correct Requirement?
-> Required Milestones Complete?
-> Required Evidence Present?
-> Completion Policy Satisfied?
-> No unresolved issue?
-> COMPLETION_VERIFIED

Step 7 - Settlement Eligibility

COMPLETION_VERIFIED
-> SETTLEMENT POLICY
-> SETTLEMENT_ELIGIBLE
-> SETTLEMENT
-> VENDOR_SETTLED

09 | DAY 7 - FAILURE & RECOVERY TESTS

SCENARIO	SIMULATION	EXPECTED CONTROL
Duplicate Payment Webhook	WEBHOOK x 3	ONE EFFECTIVE PAYMENT OUTCOME
Duplicate Completion	COMPLETION_SUBMITTED x 2	ONE EFFECTIVE COMPLETION
Duplicate Settlement	SETTLEMENT_REQUEST x 2	ONE EFFECTIVE SETTLEMENT
Vendor Cancellation	PAYMENT_CONFIRMED -> VENDOR_CANCELLED	SETTLEMENT HOLD -> CUSTOMER PROTECTION -> RECOVERY
Incomplete Service	PAYMENT_CONFIRMED -> SERVICE_INCOMPLETE	SETTLEMENT HOLD
Automation Failure	ACTION -> FAIL -> RETRY	SUCCESS OR RETRY EXHAUSTED -> DLQ / ALERT -> AUDIT

NO DOUBLE SETTLEMENT

Retries may repeat delivery, but they must not repeat the business outcome. A failed automation must never result in INR 50,000 + INR 50,000 settlement.

10 | DAY 7 - GOLDEN DEMO

Phase A - Payment Proof

Event: Corporate Conference
Requirement: Photography
Duration: 8 Hours
Commercial Value: INR 50,000

QUOTE_APPROVED
-> BOOKING_REQUEST
-> BOOKING_CREATED
-> PAYMENT_INTENT
-> RAZORPAY TEST PAYMENT
-> SERVER VERIFICATION
-> WEBHOOK
-> PAYMENT_CONFIRMED

STOP HERE

Show API / database state immediately after payment verification:

```
PAYMENT_CONFIRMED      = TRUE
COMPLETION_VERIFIED    = FALSE
SETTLEMENT_ELIGIBLE    = FALSE
VENDOR_SETTLED         = FALSE
```

Phase B - Execution + Settlement Proof

VENDOR_CONFIRMED
-> SERVICE_SCHEDULED
-> SERVICE_STARTED
-> COMPLETION_SUBMITTED
-> CORE VALIDATION
-> COMPLETION_VERIFIED
-> SETTLEMENT_ELIGIBLE
-> VENDOR_SETTLED
-> EVENT_READINESS

What This Demonstrates

- Payment success is not vendor settlement.
- Vendor execution is not completion verification.
- Completion verification is a controlled prerequisite to settlement eligibility.
- Settlement remains a Core/financial authority outcome, orchestrated safely by Nesar.

11 | UNIVERSAL ARCHITECTURE PROOF

Same Spine, Different Event Types

EVENT TYPE	REQUIREMENT TEST
Wedding	Photography

EVENT TYPE	REQUIREMENT TEST
Puja	Decoration
Corporate	Catering
Birthday	Photography

The invariant architecture

```
EVENT
  -> REQUIREMENT
  -> SERVICE LOCATION
  -> VENDOR
  -> ELIGIBILITY
  -> COST
  -> QUOTE
  -> BOOKING
  -> PAYMENT
  -> EXECUTION
  -> COMPLETION
  -> SETTLEMENT
```

What is allowed to change?

```
Event Type
  -> Requirement Template
  -> Applicability
  -> Vendor Category
  -> Category Capability
  -> Completion Policy
```

What must remain universal?

- Event Core; Requirement Core; Service Location Architecture; Vendor Ecosystem.
- Eligibility Framework; Commercial Cost Framework; Quote System; Booking Core; Payment Core.
- Automation Engine; Execution Framework; Completion Framework; Settlement Framework.
- Audit; Idempotency; Tenant Isolation.

UNIVERSALITY TEST

If a new event type requires a new Booking Engine, Payment Engine, Automation Engine, or separate backend, the architecture has failed the Day 6-7 objective.

12 | OWNERSHIP, STOP CONDITIONS & ACCEPTANCE

Final Ownership Matrix

AREA	YOGESH	NESAR	KESHAB / CORE
Event	Consume	Orchestrate	Authority
Requirement	Consume	Resolve / Orchestrate	Authority
Service Location	Vendor coverage	Context	Core context
Vendor	Owner	Consume	-
Capability	Owner	-	-
Coverage	Owner	-	-
Eligibility	Owner	-	-
Cost	Owner	-	Core data authority where applicable
Vendor Selection	Owner	-	-
Enquiry	Owner	Orchestrate	-
Quote	Owner	Orchestrate	-
Approval	Consume	Orchestrate	Authority
Booking	Consume	Request	Owner
Payment	Consume	Orchestrate	Authority
Execution	Vendor-side owner	Orchestrate	Core authority
Completion Submission	Owner	Orchestrate	-
Completion Verification	Submit evidence only	Orchestrate	Authority
Settlement	Consume result	Orchestrate	Financial authority
Audit	Domain evidence	Orchestrate	Core authority where applicable

Yogesh Stop Conditions

- Requirement contract unclear.
- Service location unavailable.
- Coverage, capacity, or availability cannot be verified.
- Pricing source is unclear or total cost cannot be validated.
- Vendor eligibility cannot be enforced.

YOGESH RULE

Stop instead of fabricating commercial data. No fake assumptions should be introduced simply to make the demo pass.

Nesar Stop Conditions

- Authorization unclear or tenant validation fails.
- Quote expired or requirement changed.
- Booking already exists.
- Payment is not verified.
- Completion is not verified.
- Settlement policy is not satisfied.

NESAR RULE

Automation must never force an unauthorized business outcome.

Final Acceptance Criteria

COMMERCIAL PROOF

Requirement -> Service Location -> Vendor Discovery -> Hard Eligibility -> Validated Total Cost -> Vendor Selection -> Enquiry -> Quote -> Approval

TRUST PROOF

Booking -> Payment Intent -> Server Verification -> Webhook -> Payment Confirmed -> Vendor Execution -> Completion Evidence -> Core Verification -> Settlement Eligibility -> Settlement

RELIABILITY PROOF

Duplicate event/approval/webhook/completion/settlement = safe. Automation failure = retry/recovery. Cross-tenant and unauthorized actions = blocked. Forged IDs = blocked. Invalid state = rejected.

THE ONE DEMO THAT MATTERS

One real commercial trust outcome, fully traceable.

CUSTOMER
-> CORPORATE EVENT
-> PHOTOGRAPHY REQUIREMENT
-> 8 HOURS
-> SERVICE LOCATION
-> ELIGIBLE VENDORS
-> VALIDATED COST
-> SELECTED VENDOR
-> ENQUIRY
-> QUOTE INR 50,000
-> CUSTOMER APPROVAL
-> BOOKING CORE
-> PAYMENT INTENT
-> RAZORPAY TEST PAYMENT
-> SERVER VERIFICATION
-> PAYMENT_CONFIRMED
-> [SETTLEMENT STILL FALSE]
-> VENDOR CONFIRMED
-> SERVICE STARTED
-> COMPLETION SUBMITTED
-> CORE VALIDATION
-> COMPLETION_VERIFIED
-> SETTLEMENT_ELIGIBLE
-> VENDOR_SETTLED
-> EVENT_READY

Universal Re-run

Wedding -> Photography
Puja -> Decoration
Corporate -> Catering
Birthday -> Photography

Same underlying commercial + execution spine.

FINAL MISSION STATEMENT - YOGESH

Build and prove the Vendor Commercial Execution layer that converts a real StarVnt requirement into a genuinely eligible, commercially validated vendor selection, enquiry and quote - without creating a second source of truth.

FINAL MISSION STATEMENT - NESAR

Build and prove the Central Orchestration layer that converts authoritative StarVnt domain events into safe, authorized, idempotent and recoverable business actions through the existing Automation Engine - without becoming the authority for Event, Vendor, Booking, Payment or Settlement.

JOINT MISSION

Prove that one universal StarVnt commercial execution spine can move a real requirement from discovery and vendor selection through customer approval, booking, verified payment, controlled vendor execution, verified completion and settlement - while remaining secure, idempotent, auditable, recoverable and independent of event type.

A Wedding is not a new system. A Puja is not a new system. A Corporate event is not a new system. A Birthday is not a new system.

They are all: EVENT -> EVENT TYPE -> REQUIREMENTS -> COMMON STARVNT EXECUTION SPINE

ENGINEERING STANDARD

Not feature-complete. Not documentation-complete. Not demo-complete. Architecture-complete enough to produce one real, traceable, repeatable commercial trust outcome.